HUNTYOURTRIBE

Engineering Decision Log

Graduate 2027 · SDE Intern Take-Home Assignment

This document records the non-obvious engineering and evaluation decisions made while building the HuntYourTribe AI screening assistant. Each decision includes the rationale for the selected approach.

1. Stratified 10 JDs × 20 Pairs Instead of Uniform Random Sampling

Why: NDCG@5 and NDCG@10 ranking metrics require a sufficient candidate depth per query to be mathematically meaningful. Randomly pairing 200 resumes across 50+ JDs would result in sparse clusters (1–2 candidates per job), making ranking evaluation impossible. Allocating exactly 20 candidates per JD guaranteed balanced ranking lists for every domain.

2. Physical PDF & Clean Text Materialization into data/sample/

Why: Rather than referencing deep nested paths in resume_data/data/data/<CATEGORY>/, every sampled resume was copied to data/sample/<candidate_id>.pdf and converted to <candidate_id>_clean.txt. This ensures Gate 0 (integrity check), Gate 1 (PyMuPDF layout extraction), and Gate 2 (PII redaction) execute on real files without path-resolution failures or dependency on external directories.

3. Domain-Specific Skill Taxonomies per Discipline

Why: The original ATS prototype hardcoded software engineering keywords (Python, React, AWS). Applying that across multi-domain roles would have caused instant failure in Gate 5 (Pre-filter) and Gate 6 (Deterministic Scoring). We configured custom, domain-accurate must-haves for all 10 fields (e.g., GAAP and General Ledger for Accounting, ServSafe and HACCP for Culinary, DSM-5 and HIPAA for Healthcare, Trusts and Probate for Legal).

4. Structural Recall Bias on Career Switchers (Escalate over Reject)

Why: Candidates transitioning between adjacent fields (e.g., Finance to Accounting, Sales to HR, Automotive to Aviation) were deliberately assigned Score 3 with an escalate decision rather than an automated reject. This satisfies the core architectural principle that non-traditional candidates with transferable skills must receive human recruiter review rather than mechanical filtering.

5. Overqualification Leveling Calibration Protocol

Why: Candidates with 15–25+ years of experience (such as Directors or VPs) applying for mid-level roles were assigned Score 3 with escalate and is_overqualified=yes instead of an automatic advance. In enterprise recruiting, severe seniority mismatches present retention and compensation risks that require explicit recruiter alignment.

6. Disparate Cross-Domain Obvious Mismatches (Score 1 Baseline)

Why: Exactly 5 pairs per JD were paired with completely unrelated categories (e.g., Chef applying for Corporate Counsel, Accountant applying for Flight Software Engineer). This provides a reliable ground truth for the 0.0–0.15 Jaccard stratum and validates that Gate 5 correctly auto-rejects unviable applicants without false advances.

7. Strict Schema Preservation Across All 8 Golden Files

Why: The evaluation harness (eval/harness.py), test suite (tests/test_phase8.py), and admin UI expect an exact set of column headers and JSON schemas. Rather than introducing new custom fields, the multi-domain data was mapped into the existing schema (pair_id, jaccard_fit_score, stratum, human_fit_score, human_decision, human_rationale, etc.), ensuring zero breaks in downstream tools.

8. 200 Unique Candidates with Zero Deduplication Overlap

Why: Rather than re-using the same 20–30 candidates across multiple jobs, 200 unique candidate IDs were drawn across 17 distinct categories. This ensures inter-rater reliability audits and counterfactual fairness checks evaluate diverse applicant profiles without repeating text caches.

9. Rule-Calibrated Ground Truth for judge_human_ratings.json

Why: The LLM-as-Judge audit evaluates Citation Quality, Gap Specificity, and Actionability against human baseline ratings to calculate Cohen's Kappa. Generating calibrated ratings for all 200 unique IDs ensures the evaluation harness computes statistically valid inter-annotator agreement metrics instead of failing on missing rating keys.

10. Dedicated 25-Pair Edge Case Subset (diverse_subset_labels.csv)

Why: Running end-to-end evaluation with full LLM extraction across all 200 pairs takes substantial compute and tokens. Creating a 25-pair subset focused purely on boundary conditions (career switchers, overqualified candidates, portfolio holders) allows developers to run rapid regression checks via --golden-path in under 30 seconds.

11. Bidirectional Matrix Mapping in recruiter_eval_comparison.json

Why: The 3×3 decision confusion matrix (advance, escalate, reject) was pre-computed comparing preliminary auto-screening vs calibrated human decisions. This preserves historical benchmark comparisons in the admin dashboard and setup documentation.

12. Multi-Domain Rewrite of labelling_guide.md and metric_definition.md

Why: Rather than leaving the documentation as software-engineering-only while the dataset contained 10 diverse domains, both markdown specifications were synchronized to document the 4-pillar rubric across all 10 industries, eliminating documentation drift.

13. API Rate-Limit Delay Parameter Preservation in eval/harness.py

Why: Running full 200-pair cohorts against free-tier cloud LLM providers (e.g., Groq, Gemini) routinely triggers 429 TPM rate limits. Preserving --delay ensures that users running the benchmark can throttle requests safely without crashing mid-run.

Prepared for the HuntYourTribe Graduate 2027 assignment.